

Quick reference

Print this. Everything on it is something you need while your hands are already on the keyboard.

The four deployables

What	Lives in	Runs on
Website, shop, console, admin	src/	Vercel → circuvent.com
Control plane	platform/	One VM, Docker Compose → api.circuvent.com
Mobile app	mobile/	Play Store · com.circuvent.app
Device firmware	firmware/	ESP32 hardware in customers' homes

Two databases, and they never meet

Shop: Neon, reached from the Next.js app, **script**, src/lib/db.ts. Control plane: Postgres on the VM, **bcrypt**, platform/api/src/db.ts. They are joined only by an SSO bridge. Assuming one database is the most common source of confusion about users.

Commands you will actually run

Task	Command	Notes
Run the site	npm run dev	One dev server at a time. Delete .next if it misbehaves.
Types	npx tsc --noEmit	Must be clean before you push.
Web tests	npm test	190 test files, including the parity guards.
Control plane	cd platform/api ; npm test	44 test files.
Control plane, live	cd platform ; docker compose up -d	Then npm run dev in platform/api.
Mobile	cd mobile ; npm run typecheck	tsc plus a dozen audits. All must pass.
Firmware	python -m platformio run	pio is not on PATH - use python -m.
This pack	npm run docs:kt	Rebuilds from the current tree.

Traps, each of which cost somebody a day

Trap	What happens	The truth
pio not found	Firmware commands fail	Use python -m platformio
Editing mobile/android/	Change vanishes later	Generated; edit app.json
Editing mobile/app.json's extra.apiBase	Nothing happens	Nothing reads it; use src/config.ts
Writing expect(x, "message")	Expect takes at most one argument	This is Jest, not Vitest
Two dev servers at once	Weird module errors	Delete .next, run one
Assuming one database	Confusion about users	Shop and control plane have separate databases and separate user tables
Adding a device type in one place	Silently no controls	A type is registered in several tables; see 07

One owner per fact

If a number lives in two places they will disagree eventually. Before you type a fact, check whether it already has a home.

Kind of fact	Owner
Anything a customer reads	src/lib/brand.ts

Kind of fact	Owner
Commercial terms, warranty length	src/lib/shop-policy.ts
What reaches hardware	src/lib/smarthome-command-map.ts
Device types and controls	Registered in several tables — see Docs/07
MQTT topic scheme	platform/api/src/config.ts
Anything about the system	Docs/ — and it is kept beside the code

When something looks broken

Symptom	Check this first
A control does nothing, no error	Parity. Is the type registered in every table, or only the one you edited?
Device state looks stale	The device is the authority. Did it republish? Frames are never persisted.
Video will not start	Is the proxy matcher excluding the relay path? Is anyone actually watching?
A scheduled job never ran	CRON_SECRET. Every cron refuses without it, and the refusal is silent.
Deployed but nothing changed	Compare the build sha from /api/health with your HEAD. Never verify by eye.
A test passes locally, fails in CI	You are probably testing a tree that includes somebody's uncommitted work.

Verify by running it, not by reading it. Nearly every defect found while this system was built appeared only when it was executed — and several were in code written an hour earlier by the person who found them. The recurring shape is a control that looks present and does nothing. They never announce themselves.